

Visual Studio Code – Análise de Aspectos Cognitivos

Pontos positivos:

Inicialmente, o programa apresenta uma necessidade alta de uma cognição reflexiva, existem muitos botões e opções possíveis. Mas, sua curva de aprendizado permite costume relativamente rápido com relação às principais funções, tornando-a mais experimental.

Possui uma divisão de espaço condizente com o objetivo do usuário, o ponto focal ao abrir o programa é sempre a tela de código, já que possui maior tamanho, ou a tela de seleção de projeto. Isso simplifica utilização e torna mais direto as interações

Segue padrões familiares de quase todas as IDEs, o que afeta diretamente certas predições, sendo nesse caso a tela de edição de código na parte central e maior, e o indexador de arquivos sempre em uma aba à esquerda.

É possível observar alguns aspectos de interoperabilidade bem implementados, a barra esquerda lateral separar módulos importantes do programa em ícones de tamanho similar, o que proporciona uma continuidade, e permite entender que fazem parte de um todo. Da mesma forma o menu superior, com opções características da maioria dos programas profissionais, “File”, “Edit” e “View” são bem familiares e também possuem certa continuidade.

Possui diferentes características de acessibilidade, por exemplo o tema de alto contraste para facilitar a percepção visual de usuários com dificuldades em distinguir cores, ou extensões para ler o código em voz.

Tem certo grau de recuperação de falhas, permite que a maioria das ações sejam refeitas para um estado anterior, principalmente por deixar o projeto integrado com versionamento GIT.

Pontos Negativos:

Certos aspectos são influenciados por um contexto não tão condizente, como exemplo, para acessar as preferencias do próprio programa VSCode, o que é essencial para muitas atividades, é necessário primeiro clicar no menu “Arquivo”, o que não é logicamente relacionado ao programa em si, e gera confusão;

Alguns aspectos de interface podem não ser tão bem interpretáveis, a parte inferior do programa reserva um pequeno espaço para diversos botões essenciais, mas que não possuem relação entre si, como por exemplo mudar o interpretador da linguagem, definir encoding, ou iniciar um live server. O que, nesse caso, viola princípios de Proximidade, Continuidade e Similaridade.

Por conta de sua ampla complexidade, necessita um arcabouço muito grande de memória de curto e longo prazo. Certas partes do programa só são acessadas em projetos específicos, o que necessitaria de uma divisão de módulos melhor delimitada para facilitar reconhecimento, mas não é o que acontece.

Na maioria das vezes para se realizar uma ação fora do escopo fundamental, o menu de contexto é essencial, mas não há nenhum botão claro, ou explicação de atalhos que permita o usuário encontra-lo intuitivamente.

Em geral, quase nenhuma das vezes os usuários irão ter um modelo mental acurado do funcionamento do programa, muitas das funcionalidades são descobertas por tentativa e busca incessante, ou algum conhecimento prévio.

Novamente, a sua complexidade também afeta a quantidade de Slips que o usuário pode realizar, devido o grande número de aspectos não explícitos na interface. A lupa na barra esquerda por exemplo, é comumente associada a pesquisa de arquivos no diretório, já que os arquivos também ficam na ala esquerda, mas na verdade a lupa é um pesquisador de palavras dentro do arquivo. Outros exemplos seguem padrão similar, que acarretam em deslizes por parte do usuário.